

The De-Facto Semantics of CriticMarkup in Markdown: A Comprehensive Analysis of Open-Source Implementations

The integration of editorial tracking directly into plain-text workflows has remained one of the most complex challenges in the evolution of lightweight markup languages. CriticMarkup, introduced as a human-readable and computer-parseable syntax for tracking additions, deletions, substitutions, highlights, and comments, provides a vital mechanism for collaborative writing in Markdown environments. The foundational CriticMarkup specification, developed by Gabe Weatherhead and Eric Hess, was published as a brief prose document. This original documentation established three governing laws: the markup must be human-readable, it must be parseable via simple regular expressions to support broad implementation, and it must remain compatible with existing Markdown and HTML syntaxes.

In practice, these three laws are in perpetual conflict. Regular expressions cannot natively manage the context-free grammars required to accurately parse nested Markdown structures. Consequently, relying on regular expressions for editorial markup fundamentally jeopardizes the integrity of the underlying Markdown document. Because the core specification intentionally left the complex edge cases of its interaction with the Markdown abstract syntax tree (AST) undefined, the ecosystem fractured. Every open-source implementation—spanning C-based command-line parsers, JavaScript-based editor overlays, Python pre-processors, and Rust crates—was forced to establish its own architectural and semantic rules.

The analysis presented in this report provides an exhaustive catalog of the existing open-source CriticMarkup ecosystem. By extracting data from primary source code, test suites, documentation, and issue tracker rulings across all major implementations, this document codifies the concrete semantic decisions made by the community. It identifies where consensus has organically emerged, where parser architectures irreconcilably diverge, and establishes a "best existing practice" profile intended to serve as a foundational blueprint for any new editor engine seeking to adopt established semantics rather than inventing its own.

Catalog of Ecosystem Implementations and Architectural Paradigms

To understand the semantic divergences in CriticMarkup, it is necessary to establish the technical landscape of the implementations that define the format's actual usage. These parsers span multiple programming languages and rely on distinctly different processing architectures, which fundamentally dictate how they resolve edge cases.

The Reference-Adjacent Pre-Processors

The earliest implementations of CriticMarkup adopted a strict pre-processor architecture,

treating editorial tracking as an independent layer applied entirely before the Markdown engine evaluates the text.

MultiMarkdown (MMD), authored in C by CriticMarkup co-creator Fletcher Penney, serves as the closest equivalent to a reference implementation. MultiMarkdown integrates CriticMarkup natively, providing command-line flags to accept (-a), reject (-r), or render the changes inline. Penney explicitly defines CriticMarkup as an "orthogonal" separate layer on top of Markdown, arguing that building a single algorithmic pass to perfectly reconcile Markdown spans crossing CriticMarkup boundaries would require a massively complex state machine, which he explicitly avoided building. MultiMarkdown executes a pure string replacement pre-pass, hunting for CriticMarkup delimiters and mutating the source string before the AST is generated.

The original CriticMarkup Toolkit, written in Python, operates on the exact same paradigm. Functioning primarily as a Command Line Interface (CLI) and via editor plugins for Sublime Text and BBEdit, the toolkit relies almost entirely on regular expression string substitution. It converts CriticMarkup blocks into HTML `<ins>` and `<del>` tags, subsequently passing the mutated string to a Markdown parser. The limitations of this approach are documented in the project's issue tracker, where reliance on raw string manipulation resulted in severe Unicode decoding errors (UnicodeDecodeError: 'ascii' codec can't decode byte) because the regex pipeline failed to handle non-ASCII text robustly.

AST Integrators and Grammars

Modern implementations have largely abandoned the regex pre-pass in favor of integrated grammar parsing, attempting to evaluate CriticMarkup simultaneously with Markdown to generate a unified abstract syntax tree.

The CodeMirror ecosystem relies heavily on lang-criticmarkup, a Lezer grammar originally developed by community contributors and heavily refined by developers like Fevol for the Obsidian "Commentator" plugin. This represents a significant architectural shift. Rather than executing string replacements, lang-criticmarkup produces distinct tree nodes (e.g., MSub for substitutions) that are integrated into the editor's live state. The implementation relies on strict test suites, including `basic_ranges.txt` for well-formedness and `malformed_ranges.txt` for parser rejection states. The Commentator plugin utilizes this AST approach to provide a WYSIWYG suggestion mode, side-panel annotations, and vault-wide metadata tracking within Obsidian. Similarly, the markdown-it-criticmarkup plugin, authored by wafer-li, implements CriticMarkup parsing as custom inline tokenization rules. Operating within the JavaScript AST pipeline, this parser evaluates `{++` and `--}` sequences as distinct tokens, pushing them into the rendering pipeline alongside standard emphasis and link tokens.

Transpilers, Wrappers, and Compilers

Several tools utilize CriticMarkup as an intermediary syntax, mapping it to format-specific macro languages for publishing.

The pancritic project is a Python-based heavy fork of the original toolkit's `criticParser_CLI.py`. Authored by ickc, it acts as a pre-processor and wrapper for Pandoc. pancritic translates CriticMarkup into either HTML `<ins>/<del>` tags or LaTeX macros (`\added{}`, `\deleted{}`, `\replaced{}`) via the LaTeX changes package. This architecture exposes a critical flaw: Pandoc's core AST generation does not process Markdown syntax located *inside* raw LaTeX macros. Thus, pancritic users experience a known limitation where Markdown within a CriticMarkup substitution fails entirely to render when exporting to PDF via LaTeX.

In the Rust ecosystem, crates such as `casile` utilize CriticMarkup to output diff highlights across Git branches for automated book generation. By comparing commits, `casile` automatically generates CriticMarkup syntax, which is then parsed to create visual syntax highlighting of additions and removals in high-resolution PDF outputs.

Editor Overlays and Interfaces

Text editors providing native CriticMarkup support must handle tokenization and live-buffer manipulation, forcing specific structural decisions.

The Emacs minor mode `cm-mode.el`, authored by `joostkremers`, utilizes the Emacs font-lock engine to highlight markup tags natively in the buffer. Because live buffer modification can easily corrupt the markup, `cm-mode` enforces strict rules, actively making the tags read-only to prevent accidental corruption. Furthermore, it introduces `cm-author` variables, automatically injecting authorship metadata directly into the `{>> <<}` comment blocks upon insertion.

The Vim plugin `vim-criticmarkup`, maintained within the `vim-pandoc` ecosystem, relies entirely on buffer string manipulation. It provides commands like `:Critic accept` and `:Critic reject` that execute automated regex deletions of the surrounding syntax markers directly in the active text buffer.

Modern academic editors, such as `Zettlr` and `Mist`, have adopted CriticMarkup to achieve feature parity with Microsoft Word's DOCX "Track Changes" functionality. `Mist` utilizes a strict round-trip serialization contract where Markdown is parsed to a rich-text state and serialized back. To prevent inline metadata bloat, `Mist` forces all extended thread metadata (author, timestamp, resolution state, colors) into YAML frontmatter, utilizing the in-text CriticMarkup comments purely as relational anchors.

Deep Analysis of Open Semantic Questions

By analyzing the source code, documentation, and bug trackers across these diverse implementations, concrete answers to the eleven critical ambiguities left open by the original specification can be codified.

1. Multi-block Spans and Boundaries

The original CriticMarkup specification claims paragraph-level support. Its documentation states: "Paragraphs may be inserted in the same manner," providing an explicit example of a `{--` tag opening in one paragraph and closing after a blank line. However, practical implementations heavily restrict this due to parser instability.

`MultiMarkdown` explicitly limits annotations, stating that CriticMarkup "must be contained within a single block (e.g. paragraph, list item, etc.). CM that spans multiple blocks will not be recognized". `Mist` enforces similarly strict line-by-line independence, warning users that "Multi-paragraph CriticMarkup is not supported. Each line is parsed independently, so a deletion that spans two paragraphs should be two separate deletions". The Emacs `cm-mode.el` explicitly advises authors "not to include newlines inside CriticMarkup tags anyway," noting that multiline font-lock matching in Emacs is highly erratic and requires users to rely on soft text wrapping rather than hard breaks.

The consensus reveals a stark divide between theoretical specification and practical engineering. While the prose specification permits multi-block spans, the de-facto standard

across robust implementations strictly confines CriticMarkup annotations to a single block or paragraph. Attempting to span blank lines reliably breaks parsers, forcing users to apply annotations line-by-line.

2. Literal Precedence and Processing Order

Because MultiMarkdown treats CriticMarkup as a pre-pass layer ("processed before any other MultiMarkdown is handled"), the engine aggressively applies CriticMarkup replacements even if the markers appear inside a fenced code block or mathematical span. This leads to unintended string destruction where code examples containing `{++` are mangled before the Markdown parser can protect them. MultiMarkdown release notes indicate constant developer interventions to mitigate this, including patches to "Fix bug in handling of '\$' delimited math" and tweaking highlight handling to prevent collisions.

Conversely, AST-integrated parsers test for the precise interaction between Markdown text and CriticMarkup grammars, generally allowing standard Markdown precedence rules to protect literal blocks. If a Markdown fenced code block tokenizes first, the inner contents are consumed as string literals, preventing any internal CriticMarkup evaluation.

The ecosystem exhibits a severe split based on parse architecture. Pre-processors aggressively prioritize CriticMarkup above all else, while modern AST parsers prioritize the Markdown code block. The modern consensus strongly favors protecting literal text; CriticMarkup inside a fenced code block must remain literal text.

3. Markdown Evaluation Inside Annotations

In pre-processor architectures like MultiMarkdown and the Original Toolkit, Markdown is absolutely parsed inside additions, deletions, and substitutions. This occurs because the pre-processor simply leaves the inner string intact or wraps it in HTML `<ins>/<del>` tags, which the subsequent Markdown engine evaluates normally on the second pass.

However, severe limitations arise based on the output target and wrapper tools. The pancritic wrapper specifically notes that "Markdown within the CriticMarkup will not be rendered in LaTeX output". Because the output is wrapped in LaTeX macros (`\deleted{...}`), Pandoc treats the block as opaque literal text rather than evaluating the Markdown nodes inside it. Mist supports inner parsing but actively strips nested block structures (like list items in blockquotes) down to flat paragraphs in the editor. Comments and highlights generally strip all inner linebreaks during HTML conversion to maintain inline validity.

Markdown inline formatting (bold, italic) inside Addition and Deletion arms is universally expected to parse and render. However, block-level Markdown inside annotations is highly unstable and generally unsupported across all architectures.

4. Delimiter Containment and State Matching

Because CriticMarkup was designed as an orthogonal layer, semantic boundaries are frequently blind to each other. MultiMarkdown explicitly warns against crossing spans: "CriticMarkup that crosses multiple MultiMarkdown (MMD) spans (e.g. `{++** foo} bar**`) will not properly manage the intended MultiMarkdown markup. This example would not result in bold being applied".

In modern AST parsers, delimiter matching states (such as those for strong/emphasis) are strictly contained within the bounds of their parent node. A substitution node (MSub) in a Lezer grammar generates distinct text contexts for the "from" and "to" branches. Matching an asterisk

inside the `{--` arm to an asterisk in the `{++` arm is fundamentally impossible in a hierarchical AST because the text nodes do not share a parent scope capable of linking the formatting tokens. Delimiter matching states are strictly confined. A Markdown tag opened inside a CriticMarkup annotation must be closed inside that exact same annotation arm. Overlapping spans are universally treated as malformed edge cases.

5. Nesting and Recursion

The original specification remains entirely silent on whether annotations can nest. MultiMarkdown v5 explicitly introduced "recursive support for CriticMarkup syntax (e.g. an addition can be included inside a substitution)". However, `pancritic` explicitly warns that "Nesting CriticMarkup might have unexpected behavior, especially in LaTeX output," directing users to the specification's caveats.

The Lezer grammar for `lang-criticmarkup` was specifically forked by the developer of the Commentator plugin to resolve "nesting issues of the original grammar," attempting to allow valid recursion within the CodeMirror environment.

Nesting is a highly sought-after power-user feature, specifically the act of embedding Comments inside Additions or Deletions to explain a change. True recursive evaluation is successfully supported by MultiMarkdown and modern ASTs, but generally triggers catastrophic failures in older regex pipelines and LaTeX transpilers.

6. Accept/Reject Semantics and Output Generation

When accepting or rejecting changes, tools universally operate on the raw source string rather than the parsed tree. MultiMarkdown's `-a` (accept) and `-r` (reject) command-line flags execute a pre-parsing string manipulation to output a clean "new" or "original" document prior to HTML rendering. Vim's `:Critic accept` command directly deletes the rejected text and the delimiter strings from the active text buffer. Mist modifies the serialized Markdown string on export, erasing the tags depending on the user's resolution choices (e.g., "Reject addition: the text is removed").

Because the accept/reject action takes place before or during AST serialization, any resulting structural changes to the Markdown—such as accepting an addition that contains a heading `#` marker—are perfectly re-parsed as valid Markdown by the subsequent engine cycle.

Accept/Reject operations are universally strict string manipulations. They produce clean Markdown source text devoid of CriticMarkup markers, allowing the underlying Markdown engine to evaluate the resulting structural text entirely from scratch.

7. Highlight-Comment Pairing and Anchoring

The original spec provides a specific syntax for pairing comments with highlights: `{==highlighted text==}{>>comment<<}`. In rich editor implementations, this creates a complex data-binding challenge. Emacs `cm-mode.el` allows a user to invoke the comment command on an active region, automatically wrapping it in a highlight and attaching the comment adjacent to it. In the Mist and Zettlr implementations, this adjacent syntax is split into two distinct Markdown ranges on import. The editorial thread is linked to the highlighted passage by matching a generated frontmatter ID in the YAML metadata to the in-text comment string. If a comment exists without a highlight (`{>>A note<[span_107](start_span)[span_107](end_span)<}`), Mist treats it as a zero-width point marker in the document.

A highlight immediately followed by a comment is the universally recognized pairing syntax. Standalone comments act as inline blockless point-anchors, while rich editors increasingly rely on frontmatter to bridge the in-text anchor to external metadata.

8. Escaping and Literal Markers

The original CriticMarkup specification provides absolutely no escape mechanism for the syntax. Because MultiMarkdown evaluates CriticMarkup before its own escaping engine runs, placing a backslash before the syntax (e.g., `\{++`) generally fails to prevent the pre-processor from capturing it and initiating string replacement.

In Issue 33 of the CriticMarkup Toolkit repository, discussions regarding rendering encoding crashes demonstrate that the parsing script evaluates the text as a monolithic string block, providing no logic for ignoring escaped sequences.

There is no universally supported native escape sequence for CriticMarkup. Authors must rely on Markdown's inline code spans (`{++``) to escape markers, assuming the specific implementation's precedence rules allow code spans to protect inner strings from the CriticMarkup parser.

9. Structural Interactions

Because CriticMarkup operates conceptually at the inline text level, introducing block-level modifiers inside annotations reliably breaks table and list parsers. Mist explicitly forbids complex structural logic inside CriticMarkup, noting that lists inside edits will flatten upon export.

MultiMarkdown v6 adjusted its handling of table captions and HTML blocks to be more resilient, but overlapping an inline `{++` block with a GFM table pipe `|` will irreparably break the table's column alignment once the Markdown parser takes over, as the pipe is consumed or relocated during string replacement.

However, CriticMarkup interfaces gracefully with YAML frontmatter. Modern applications like Mist leverage frontmatter specifically to store extended comment metadata without polluting the inline Markdown body, establishing a clear separation of concerns.

CriticMarkup is toxic to GFM table syntax if delimiters contain structural characters. It is established practice to keep CriticMarkup entirely inside table cells, never wrapping across cell boundaries. Storing rich track-changes metadata in YAML frontmatter is the modern standard for maintaining clean document bodies.

10. Unclosed Markers and Editor State

In static site generators and command-line compilers (like Pandoc or the MultiMarkdown CLI), an unclosed `{++` is safely treated as literal text, as the regular expression simply fails to find the closing boundary.

However, in live-updating AST editors like CodeMirror 6 utilizing lang-criticmarkup, an unclosed opening bracket creates a catastrophic matching failure. The grammar documentation explicitly notes that an unclosed `{++` "will match all subsequent tokens, and thus later ranges might not get parsed correctly," effectively blinding the syntax highlighter for the entire remainder of the document until the user types the closing bracket.

Unclosed markers are a critical vulnerability in continuous AST parsers. While regex gracefully ignores them, integrated grammars swallow the document. Live editors must enforce strict fallback limits (e.g., terminating an unclosed search at the end of the paragraph) to prevent

document-wide rendering failures during live typing.

11. Parse Architecture and Systemic Bugs

The architecture chosen dictates the bug profile of the implementation. MultiMarkdown and the Python toolkit employ a pure regex pre-pass. This leads to order-of-operations bugs, such as the accidental destruction of math spans and code blocks. pancritic employs a post-processing wrapper architecture, mapping string tokens to LaTeX macros. This leads to evaluation bugs, as Pandoc refuses to parse Markdown trapped inside LaTeX. CodeMirror and markdown-it-criticmarkup employ integrated single-pass grammars. This leads to state-matching bugs, where unclosed tags corrupt the entire tree.

The Formal Standardization Efforts and the CommonMark Debates

Beyond the original prose specification, formal standardization efforts have largely occurred in the margins of broader Markdown communities. The most robust technical formalization exists in the lang-criticmarkup Lezer specification, which explicitly defines the AST nodes for CodeMirror 6 integration. Furthermore, proposals by Fevol (Commentator) outline specific extensions to the metadata separator and backwards compatibility rules.

The most extensive philosophical debate regarding CriticMarkup occurred within the CommonMark standardization community under topics evaluating "Reviewing Markups" and "Strikethrough" syntaxes [cite: 85-102]. The core conflict revolved around the semantic difference between public-facing rhetorical text and internal editorial review.

When users requested the addition of the `~~strikethrough~~` syntax to the CommonMark core spec, developers immediately recognized semantic collisions. As forum contributor chrisalley observed, HTML5 draws a sharp distinction between the `<s>` element (representing text that is no longer accurate or relevant) and the `<del>` element (representing an editorial removal from the document). GitHub Flavored Markdown (GFM) famously maps `~~` to `<del>`, creating widespread semantic confusion regarding intent.

Contributors argued that for true editorial review (additions, deletions, comments), the syntax needed to be distinctly separate from standard formatting. CriticMarkup's `{-- --}` and `{++ ++}` syntaxes were repeatedly cited as the de-facto standard for this workflow. John MacFarlane (jgm), the creator of Pandoc and heavily involved in CommonMark, noted significant parser-level concerns regarding CriticMarkup. Specifically, MacFarlane questioned how cross-paragraph ranges would interact with Markdown's fundamental two-level concept of blocks versus inlines, noting that "you might e.g. want to strikeout part of a code block—but of course we wouldn't support markup in a code block in any case".

Other proposals attempted to invent hybrid syntaxes like `>>~~deleted text~~<<` or utilizing HTML comments `<!-- -->` and Pandoc's `---` YAML blocks for hidden comments. However, the community continually deferred back to CriticMarkup for "Reviewing markups." Despite the heavy reliance on CriticMarkup for collaborative drafting, no official CommonMark extension was ever codified. The CommonMark consensus remains that CriticMarkup should operate as an independent, unofficial extension to preserve the simplicity of the core parser.

Decision Matrix: Ecosystem Behaviors

The following matrix synthesizes how the primary implementations answer the core semantic questions across the ecosystem.

Feature / Question	MultiMarkdown (C)	Toolkit / pancritic (Python)	lang-criticmarkup (JS/TS)	Mist / Zettlr / Editor Overlays
Multi-block spans	No. Confined to a single block/paragraph.	Yes (in spec). Paragraphs can be inserted.	Undefined. Fails across large blocks.	No. Multi-paragraph explicitly unsupported.
Literal precedence	CM wins. Processed before code/math spans.	CM wins. Executed via Regex pre-pass.	Markdown wins. AST protects code blocks.	Markdown wins. Requires AST protection.
MD inside annotations	Yes , evaluated post-string replacement.	Yes , but fails in LaTeX outputs via pancritic.	Yes , via AST node nesting contexts.	Yes , but restricted to flat paragraphs.
<i>Delimiter containment</i>	Strict. Cross-span matching breaks parsing.	Undefined. Relies entirely on post-processor.	Strict. AST nodes isolate contexts.	Strict. Inline state resets at boundary.
<i>Nesting recursion</i>	Yes. Recursive support added in v5.	Buggy. LaTeX outputs fail on nested input.	Yes. Explicitly patched to handle MSub nodes.	Undefined. Fails in YAML round-trips.
Accept/Reject result	Source string modification (-a / -r flags).	String to HTML/LaTeX macro translation.	AST tree manipulation.	Source string serialization.
Highlight-Comment	Yes , parses sequentially.	Yes , standard attached pair.	Yes , sequential nodes.	Yes , paired via frontmatter ID matching.
Escaping markers	None natively. Fails inside code due to precedence.	None natively. Strict string evaluation.	Markdown code blocks protect syntax.	Markdown code blocks protect syntax.
Structural conflicts	Breaks tables/lists if boundaries cross.	Breaks LaTeX macros if MD is inside.	Standard block failure across nodes.	Uses frontmatter safely for metadata.
<i>Unclosed markers</i>	Ignored (treated as literal text by regex).	Ignored (regex fails to match).	Catastrophic. Swallows subsequent tokens.	Syncs keystrokes, limits to line bounds.
Parse architecture	Pure regex/string pre-pass.	Regex pre-pass / Pandoc AST Wrapper.	CodeMirror 6 Lezer AST grammar.	Dual-parse serialization (AST to YAML).

Recommended "Best Existing Practice" Profile

For a modern editor engine seeking to adopt CriticMarkup while maximizing ecosystem compatibility and minimizing catastrophic parsing failures, it is critical to abandon the historical

"regex pre-pass" architecture. While MultiMarkdown and the original Python toolkit utilized regular expressions, modern Markdown environments rely strictly on Context-Free Grammars and unified abstract syntax trees. A new implementation should adopt the following behavioral profile to match established best practices:

1. **Confine to Single Blocks:** Enforce a strict single-block limitation for all annotations. Multi-line or multi-paragraph annotations must be represented by multiple consecutive tags. Accepting this limitation prevents AST corruption and mirrors the restrictions required by Mist, Emacs cm-mode, and MultiMarkdown.
2. **Prioritize Literal Markdown:** Process Markdown fenced code blocks, inline code spans, and mathematical equations *before* evaluating CriticMarkup. A `{++}` inside a code block must remain a literal string. This resolves the persistent string-destruction bugs that plagued early regex pipelines.
3. **Evaluate Inline Markdown Internally:** Permit inline Markdown (bold, italic, links) inside Addition and Deletion arms. Conversely, strictly disable block-level Markdown (headers, blockquotes, lists) inside annotations to prevent tree corruption and flattening during export.
4. **Enforce Strict Delimiter Containment:** Never allow an emphasis or link marker to open inside a CriticMarkup annotation and close outside of it. The parser state must hard-reset at the boundary of a CriticMarkup arm, treating overlapping spans as plain text to prevent rendering cross-contamination.
5. **Support Shallow Nesting:** Allow Comments `{>>...<<}` to nest inside Additions and Deletions, as this is a heavily utilized workflow for explaining editorial rationale. Disallow deep recursive nesting (e.g., Substitutions inside Substitutions) to preserve parser speed and maintain readability.
6. **Operate Accept/Reject on Source:** Accept and Reject operations must execute as string-level modifications to the raw document, immediately followed by a complete re-parse of the Markdown AST. This guarantees that structural changes (like deleting a list marker) are correctly re-evaluated by the core Markdown engine.
7. **Pair Highlights and Comments via Metadata:** Treat `{==highlight==}{>>comment<<}` as a single bound entity. For rich editor environments, parse these nodes out to a sidebar and store extended metadata (author, time, resolution state) in the document's YAML frontmatter, utilizing a unique ID mapping to keep the document body clean.
8. **Rely on Code Spans for Escaping:** Do not invent a custom backslash escape syntax (e.g., `\{++}`). Force users to use standard Markdown inline code spans (``{++}``) to write literal markers, relying on the literal precedence rule to protect the string.
9. **Limit Unclosed Marker Search Space:** To prevent the "document swallowing" bug seen in Lezer grammars, limit the lookahead for a closing marker to the next double-newline (`\n\n`). If a closing marker is not found within the current paragraph block, gracefully degrade the unclosed opening marker to literal text to protect the remainder of the document's AST.
10. **Isolate from Structural Table Syntax:** Treat GFM table pipes (`|`) located inside a CriticMarkup annotation as literal text belonging to the annotation, not as table column dividers. This prevents mid-edit track changes from collapsing the entire table architecture during live preview.
11. **Avoid LaTeX Macro Wrapping:** When compiling to PDF or LaTeX, avoid the pancratic architecture of wrapping CriticMarkup directly in `\added{}` macros if the content contains nested Markdown. An intermediate AST resolution pass must convert the Markdown to LaTeX *before* wrapping it in the change-tracking macro.

By adhering to this synthesized profile, a new editor engine will bypass the systemic ordering bugs that plagued early regex-based implementations, while natively aligning with the operational expectations of users migrating from MultiMarkdown, Obsidian, Zettlr, and Pandoc.

Works cited

1. Various tools to use CriticMarkup in your daily workflow - GitHub, <https://github.com/CriticMarkup/CriticMarkup-toolkit>
2. DivineDominion/criticmarkup.tmbundle: A CriticMarkup bundle for TextMate v2 - GitHub, <https://github.com/DivineDominion/criticmarkup.tmbundle>
3. Releases · fletcher/MultiMarkdown-5 - GitHub, <https://github.com/fletcher/MultiMarkdown-5/releases>
4. GitHub - fletcher/MultiMarkdown-6: This project is now deprecated. Please use MultiMarkdown-7 instead!, <https://github.com/fletcher/MultiMarkdown-6>
5. GitHub - fletcher/MultiMarkdown-7: Lightweight markup processor to produce HTML, LaTeX, and more. (Currently in pre-release.), <https://github.com/fletcher/MultiMarkdown-7>
6. CriticMarkup, <https://fletcher.github.io/MultiMarkdown-5/criticmarkup.html>
7. MultiMarkdown-4/Makefile at master · fletcher ... - GitHub, <https://github.com/fletcher/MultiMarkdown-4/blob/master/Makefile>
8. Doesn't work with cyrillic texts #33 - GitHub, <https://github.com/CriticMarkup/CriticMarkup-toolkit/issues/33>
9. lang-criticmarkup/README.md at main · nathanlesage/lang ... - GitHub, <https://github.com/nathanlesage/lang-criticmarkup/blob/main/README.md>
10. Criticmarkup grammar for the lezer parser - GitHub, <https://github.com/Fevol/criticmarkup-parser>
11. Beta Plugin: Commentator - Suggestions and Comments with CriticMarkup, <https://forum.obsidian.md/t/beta-plugin-commentator-suggestions-and-comments-with-criticmarkup/66013>
12. wafer-li/markdown-it-criticmarkup - GitHub, <https://github.com/wafer-li/markdown-it-criticmarkup>
13. GitHub - ickc/pancritic: using criticmarkup in the pandoc markdown source, <https://github.com/ickc/pancritic>
14. pancritic · PyPI, <https://pypi.org/project/pancritic/0.1/>
15. casile - crates.io: Rust Package Registry, <https://crates.io/crates/casile>
16. Emacs minor mode for handling CriticMarkup. - GitHub, <https://github.com/joostkremers/criticmarkup-emacs>
17. vim-pandoc/vim-criticmarkup - GitHub, <https://github.com/vim-pandoc/vim-criticmarkup>
18. vim-pandoc - GitHub, <https://github.com/vim-pandoc>
19. Workflow for handling track changes and comments from exported docx back to zettlr #3337, <https://github.com/Zettlr/Zettlr/discussions/3337>
20. markdown-and-criticmarkup.md - inanimate-tech/mist - GitHub, <https://github.com/inanimate-tech/mist/blob/main/docs/markdown-and-criticmarkup.md>
21. Releases · fletcher/MultiMarkdown-6 - GitHub, <https://github.com/fletcher/MultiMarkdown-6/releases>
22. Feature Request: Strikethrough text - Extensions - CommonMark Discussion, <https://talk.commonmark.org/t/feature-request-strikethrough-text/220>
23. Strikeout threw out? Strikethrough strikes out, throughout? - #34 by chrisalley - Extensions, <https://talk.commonmark.org/t/strikeout-threw-out-strikethrough-strikes-out-throughout/820/34>
24. Strikeout threw out? Strikethrough strikes out, throughout? - Page 2 - Extensions - CommonMark Discussion, <https://talk.commonmark.org/t/strikeout-threw-out-strikethrough-strikes-out-throughout/820?page=2>
25. Reviewing Markups - Extensions - CommonMark Discussion, <https://talk.commonmark.org/t/reviewing-markups/824>
26. Strikeout threw out? Strikethrough strikes out, throughout? - CommonMark Discussion, <https://talk.commonmark.org/t/strikeout-threw-out-strikethrough-strikes-out-throughout/820>
- 27.

Feature Request: underline text - #10 by mofosyne - Spec - CommonMark Discussion,
<https://talk.commonmark.org/t/feature-request-underline-text/343/10> 28. Strikeout threw out?
Strikethrough strikes out, throughout? - #44 by,
<https://talk.commonmark.org/t/strikeout-threw-out-strikethrough-strikes-out-throughout/820/44>
29. Strikeout threw out? Strikethrough strikes out, throughout? - Page 3 - Extensions -
CommonMark Discussion,
<https://talk.commonmark.org/t/strikeout-threw-out-strikethrough-strikes-out-throughout/820?page=3> 30. "comment" facility in commonmark? - Extensions,
<https://talk.commonmark.org/t/comment-facility-in-commonmark/1271>